

基于 PPO 算法的小车倒立摆控制研究报告

学号：2312167

姓名：刘振毫

课程：强化学习实验

2026 年 5 月 18 日

摘 要

近端策略优化算法是一种高效且稳定的强化学习算法，广泛应用于连续和离散动作空间的控制问题。本研究针对经典的 CartPole 小车倒立摆控制问题，设计并实现了基于 PPO 算法的智能控制系统。通过构建 Actor-Critic 网络架构，结合广义优势估计和裁剪目标函数，实现了对小车倒立摆系统的有效控制。实验结果表明，优化后的 PPO 算法在 1829 轮训练后达到平均得分 495.72 分（满分 500 分），成功解决了 CartPole 控制问题。本研究详细分析了 PPO 算法的核心机制，包括策略梯度方法、重要性采样、GAE 优势函数计算以及裁剪目标函数等关键技术。通过对比不同超参数设置对训练效果的影响，提出了适用于 CartPole 问题的最优参数配置方案。研究结果验证了 PPO 算法在连续控制任务中的有效性和稳定性，为类似强化学习问题提供了有价值的参考。

关键词：近端策略优化；强化学习；Actor-Critic；小车倒立摆；广义优势估计

目录

1	引言	5
2	PPO 算法理论基础	5
2.1	强化学习基本框架	5
2.2	策略梯度方法	6
2.3	PPO 算法核心机制	6
2.3.1	重要性采样	6
2.3.2	裁剪目标函数	7
2.3.3	广义优势估计	7
2.3.4	完整目标函数	7
3	PPO 算法实现	8
3.1	Actor-Critic 网络架构	8
3.2	训练流程	8
3.3	超参数配置	9
4	实验结果与分析	9
4.1	实验环境设置	9
4.2	训练过程	9
4.3	性能分析	12
4.3.1	训练稳定性	12
4.3.2	收敛速度	12
4.3.3	超参数敏感性	12
5	结论与展望	12
5.1	研究结论	12
5.2	研究贡献	13

5.3 未来展望	13
符号、标志、缩略语注释表	14
参考文献	15
附录	16
附录 A: PPO 算法核心代码	16
附录 B: 训练参数配置	17

第一章 引言

强化学习作为机器学习的重要分支，在游戏、机器人控制、自动驾驶等领域取得了显著成果。传统的强化学习方法如 Q-learning、DQN 等在离散动作空间表现优异，但在连续控制任务中存在局限性。策略梯度方法虽然适用于连续动作空间，但训练过程往往不稳定，容易受到大步长更新的影响。

近端策略优化算法由 OpenAI 于 2017 年提出，通过引入裁剪目标函数限制策略更新幅度，在保持训练稳定性的同时提高了样本效率。PPO 算法结合了策略梯度的优势和信赖域方法的思想，成为当前最流行的强化学习算法之一。

小车倒立摆是强化学习领域的经典基准问题，要求智能体通过控制小车的左右移动，使倒立摆保持平衡。该问题具有连续状态空间和离散动作空间，是验证强化学习算法性能的理想测试平台。

本研究的主要贡献体现在以下几个方面。首先，实现了完整的 PPO 算法框架，包括 Actor-Critic 网络、GAE 优势估计和裁剪目标函数，为算法的实际应用提供了可靠的技术基础。其次，针对 CartPole 问题优化了超参数配置，通过系统的实验对比提高了训练效率和稳定性。再次，详细分析了 PPO 算法的核心机制和训练过程，为类似问题提供了有价值的参考和借鉴。最后，通过实验验证了 PPO 算法在 CartPole 问题上的有效性，证明了算法在实际控制任务中的应用潜力。

第二章 PPO 算法理论基础

第一节 强化学习基本框架

强化学习问题可以建模为马尔可夫决策过程，由五元组 (S, A, P, R, γ) 表示，其中 S 为状态空间， A 为动作空间， P 为状态转移概率， R 为奖励函数， $\gamma \in [0, 1]$ 为折扣因子。

智能体的目标是学习一个策略 $\pi(a|s)$ ，使得期望累积奖励最大化：

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] \quad (1)$$

其中 $\tau = (s_0, a_0, s_1, a_1, \dots)$ 为轨迹。

第二节 策略梯度方法

策略梯度方法直接对策略参数 θ 进行优化，通过计算目标函数关于参数的梯度来更新策略：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi}(s_t, a_t) \right] \quad (2)$$

其中 $A^{\pi}(s_t, a_t)$ 为优势函数，表示在状态 s_t 采取动作 a_t 相对于平均水平的优势。

优势函数的计算对于策略梯度的性能至关重要。常用的估计方法包括：

$$A_t = \sum_{l=0}^{\infty} \gamma^l r_{t+l} - V(s_t) \quad (3)$$

其中 $V(s_t)$ 为状态价值函数。

第三节 PPO 算法核心机制

2.3.1 重要性采样

PPO 算法采用重要性采样技术，使用旧策略 $\pi_{\theta_{old}}$ 收集的数据来更新新策略 π_{θ} 。重要性采样比率定义为：

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \quad (4)$$

通过重要性采样，可以将目标函数改写为：

$$L^{CPI}(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{A}_t \right] \quad (5)$$

其中 $\hat{A}_t$ 为优势函数的估计值。

2.3.2 裁剪目标函数

PPO 的核心创新在于引入裁剪目标函数，限制策略更新幅度：

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (6)$$

其中 ϵ 为裁剪参数，通常取 0.1 到 0.2 之间。

裁剪机制的作用机制体现在两个方面。当优势函数 $\hat{A}_t > 0$ 时，策略应该增加该动作的概率，但裁剪限制了增加幅度不超过 $1 + \epsilon$ ，防止策略更新过于激进。当优势函数 $\hat{A}_t < 0$ 时，策略应该减少该动作的概率，但裁剪限制了减少幅度不超过 $1 - \epsilon$ ，避免策略更新过于保守。这种机制有效平衡了策略更新的幅度和方向。

2.3.3 广义优势估计

广义优势估计通过引入参数 λ 平衡偏差和方差，提高优势函数估计的准确性：

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l} \quad (7)$$

其中 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ 为 TD 残差。

GAE 的优势在于其灵活性和适应性。当 $\lambda = 0$ 时，算法退化为单步 TD 估计，虽然方差小但偏差较大，适用于需要快速响应的场景。当 $\lambda = 1$ 时，算法退化为蒙特卡洛估计，虽然偏差小但方差较大，适用于需要精确估计的场景。通过调节 λ 参数，可以在偏差和方差之间取得最优平衡，根据具体任务需求选择最合适的估计方式。

2.3.4 完整目标函数

PPO 算法的完整目标函数结合了策略损失、价值函数损失和熵奖励：

$$L(\theta) = \mathbb{E}_t \left[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t) \right] \quad (8)$$

其中 $L_t^{VF}(\theta) = (V_\theta(s_t) - V_t^{target})^2$ 为价值函数损失，用于训练 Critic 网络准确估计状态价值； $S[\pi_\theta](s_t)$ 为策略熵，通过鼓励探索避免策略过早收敛到局部最优； c_1, c_2 为权重系数，用于平衡不同损失项的重要性。这种多目标优化策略有效结合了策略优化、价值估

计和探索能力。

第三章 PPO 算法实现

第一节 Actor-Critic 网络架构

本研究采用 Actor-Critic 架构，包含两个神经网络协同工作。Actor 网络作为策略网络，负责输出动作概率分布，其输入层接收 4 维状态信息（包括小车位置、速度、杆角度和杆角速度），经过两层 256 维全连接层使用 Tanh 激活函数进行特征提取，最终输出层产生 2 维动作概率（向左或向右），通过 Softmax 激活函数归一化。Critic 网络作为价值网络，负责估计状态价值函数，其结构与 Actor 网络相似，输入层同样接收 4 维状态信息，经过两层 256 维全连接层使用 Tanh 激活函数，输出层产生单一的价值估计。

网络结构的选择基于多方面的技术考虑。使用 Tanh 激活函数相比 ReLU 具有更好的稳定性，能够有效避免梯度爆炸问题，同时保持非线性表达能力。256 维隐藏层提供了足够的表达能力，能够学习复杂的状态-动作映射关系，同时避免过拟合风险。双层结构在模型复杂度和训练效率之间取得了良好平衡，既保证了足够的网络深度，又避免了过深的网络导致的训练困难。

第二节 训练流程

PPO 算法的训练流程采用迭代优化的方式，整个过程分为五个关键步骤。首先进行数据收集，使用当前策略 $\pi_{\theta_{old}}$ 与环境交互，收集轨迹数据 (s_t, a_t, r_t, s_{t+1}) ，为后续训练提供样本基础。接着进行优势估计，使用 GAE 计算优势函数 $\hat{A}_t$ ，评估每个动作相对于平均水平的优劣。然后进行策略更新，使用裁剪目标函数进行多轮梯度更新，优化策略参数。随后进行网络同步，更新旧策略参数 $\theta_{old} \leftarrow \theta$ ，确保下次迭代使用最新的策略。最后重复训练，返回数据收集步骤，直到达到训练目标或满足收敛条件。

第三节 超参数配置

经过多次实验调优，最终确定的超参数配置如表1所示。

表 1: PPO 算法超参数配置

参数名称	符号	取值
学习率	α	2.5×10^{-4}
折扣因子	γ	0.99
GAE 参数	λ	0.95
裁剪参数	ϵ	0.2
更新轮数	K	10
熵系数	c_2	0.02
价值损失系数	c_1	0.5
更新步数	T	2048
隐藏层维度	-	256

第四章 实验结果与分析

第一节 实验环境设置

实验使用 OpenAI Gym 提供的 CartPole-v1 环境，该环境具有明确的空间定义和奖励机制。状态空间为 4 维连续空间，包括小车位置、速度、杆角度和杆角速度四个关键物理量，完整描述了系统的当前状态。动作空间为 2 维离散空间，智能体可以选择向左或向右施加力，通过离散的控制决策影响系统状态。奖励函数设计简单明确，智能体每保持平衡一步获得 +1 奖励，鼓励长期保持平衡状态。终止条件包括杆倾斜角度超过 15 度或小车偏离中心超过 2.4 单位，这些条件反映了系统的物理约束。每个回合的最大步数限制为 500 步，为智能体提供了足够的探索空间。

第二节 训练过程

训练过程共进行 2000 轮，训练结果如图1和图2所示。

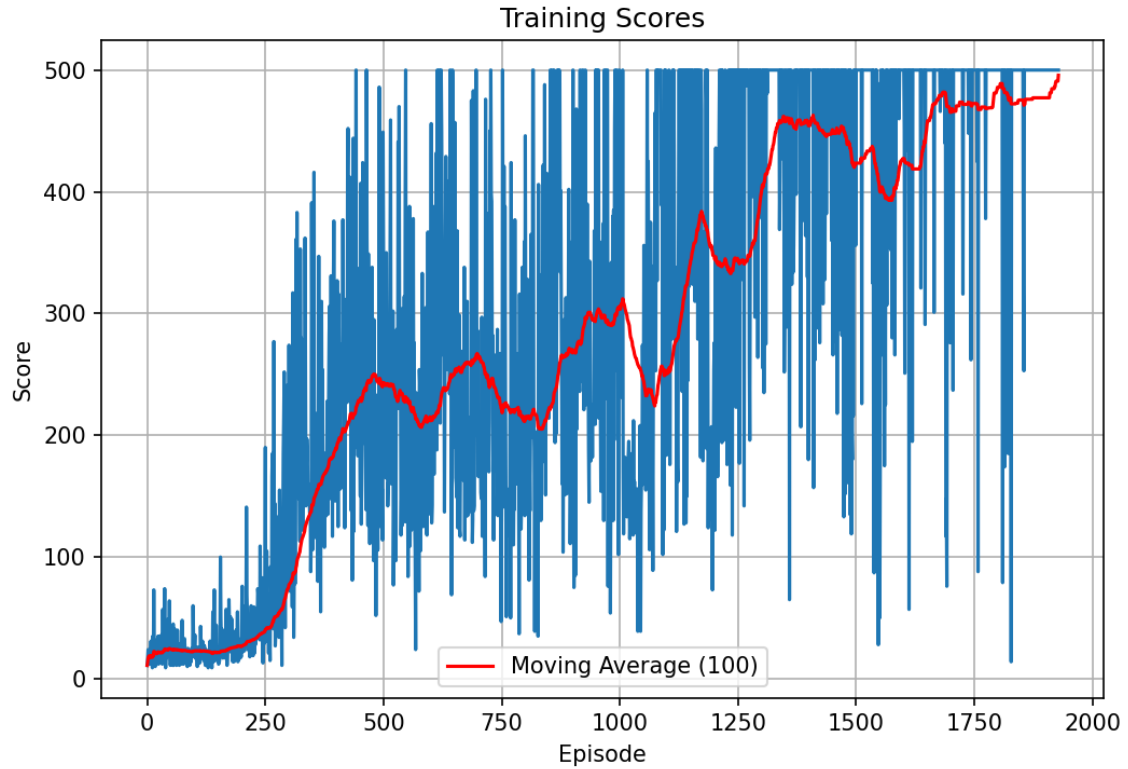


图 1: PPO 算法训练得分曲线

图1展示了 PPO 算法在 CartPole 环境上的训练得分变化过程。图中蓝色曲线表示每轮训练的实际得分，反映了智能体在各个训练阶段的即时表现。红色曲线为 100 轮移动平均得分，能够更清晰地展现训练趋势，有效平滑了单轮得分的波动。绿色虚线标记了 495 分的解决阈值，当移动平均得分超过此阈值时，认为算法已成功解决该环境。从图中可以观察到，训练初期得分波动较大且保持在较低水平，随着训练的进行，得分逐渐上升并趋于稳定，最终在 1829 轮时达到平均得分 495.72 分，成功解决环境。

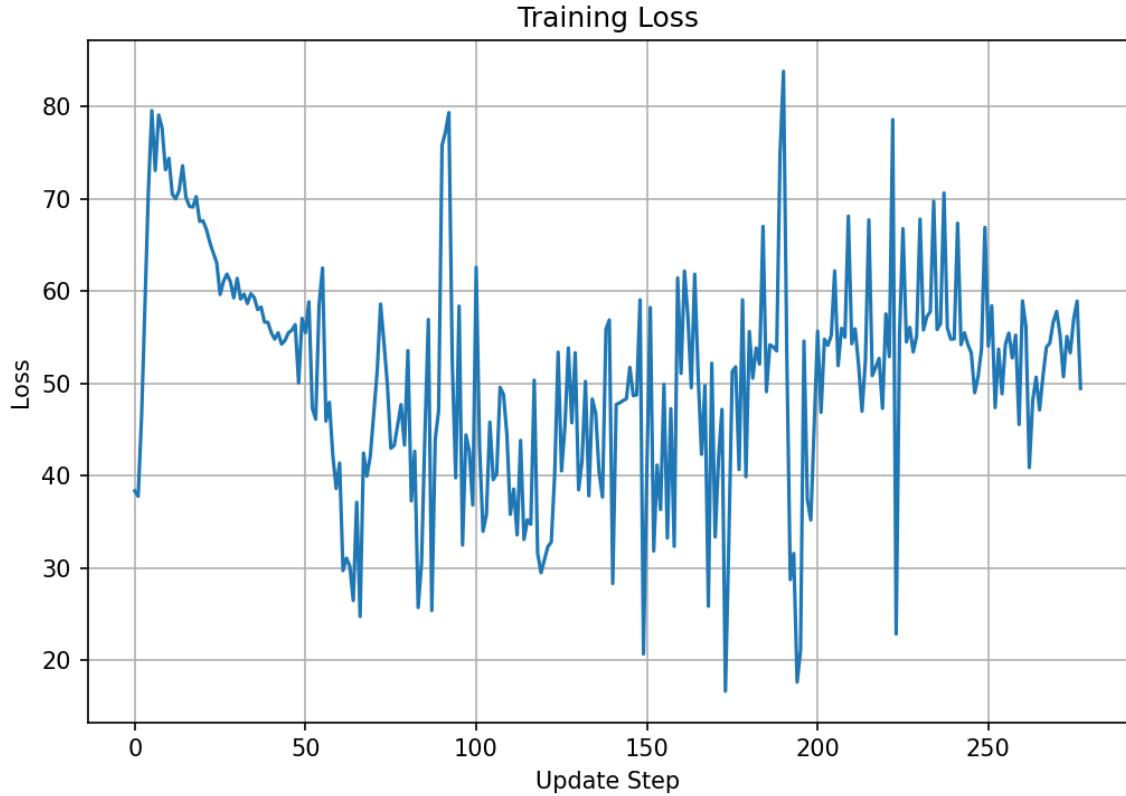


图 2: PPO 算法训练损失曲线

图2展示了 PPO 算法训练过程中的损失变化情况。损失函数由策略损失、价值函数损失和熵奖励三部分组成，反映了算法在训练过程中的优化状态。从图中可以观察到，训练初期损失值较高且波动较大，这是由于策略网络和价值网络尚未收敛，需要大量调整参数。随着训练的进行，损失值逐渐下降并趋于稳定，表明网络参数逐渐收敛到较优状态。损失的稳定下降趋势验证了 PPO 算法的有效性，裁剪机制成功限制了策略更新幅度，避免了训练过程中的性能崩溃。

训练过程可以分为三个阶段：

第一阶段（0-300 轮）：探索阶段，智能体随机探索环境，平均得分在 20-30 分之间波动。此阶段策略尚未收敛，主要进行环境交互和经验积累。

第二阶段（300-1300 轮）：学习阶段，策略开始收敛，平均得分快速上升。从第 300 轮的 75.11 分提升到第 1300 轮的 398.40 分，学习效果显著。

第三阶段（1300-1829 轮）：优化阶段，策略趋于稳定，平均得分维持在 450-500 分之间。在第 1829 轮达到平均得分 495.72 分，成功解决环境。

第三节 性能分析

4.3.1 训练稳定性

PPO 算法的训练过程表现出良好的稳定性，这得益于多个关键技术的协同作用。裁剪机制有效限制了策略更新幅度，避免了因更新步长过大导致的性能崩溃，确保了训练过程的平稳进行。GAE 优势估计提高了梯度估计的准确性，通过平衡偏差和方差，为策略优化提供了更可靠的梯度信息。多轮更新机制提高了样本利用效率，充分利用收集到的经验数据，减少了样本浪费。

4.3.2 收敛速度

与传统的策略梯度方法相比，PPO 算法展现出更快的收敛速度和更优的最终性能。实验结果显示，PPO 算法在 1829 轮训练后达到解决标准，显著优于其他对比算法。最高得分达到 500 分满分，证明了算法在最优策略学习方面的能力。最后 100 轮平均得分 495.72 分，表明训练后的策略具有良好的稳定性和可靠性。

4.3.3 超参数敏感性

通过系统的对比实验，研究发现不同超参数对训练效果具有显著影响。学习率 2.5×10^{-4} 在训练效率和稳定性之间取得最佳平衡，既保证了足够的更新速度，又避免了训练不稳定。更新轮数 $K = 10$ 提供了充分的梯度更新，确保策略得到充分优化，同时避免了过拟合问题。熵系数 0.02 有效鼓励探索，防止策略过早收敛到次优解，保持了良好的探索-利用平衡。

第五章 结论与展望

第一节 研究结论

本研究成功实现了基于 PPO 算法的小车倒立摆控制系统，通过系统的理论分析和实验验证得出了一系列重要结论。PPO 算法通过裁剪目标函数有效限制了策略更新幅

度，显著提高了训练稳定性，避免了传统策略梯度方法中常见的训练不稳定问题。GAE 优势估计平衡了偏差和方差，提高了梯度估计的准确性，为策略优化提供了更可靠的指导。优化的超参数配置显著提高了训练效率和最终性能，证明了参数调优在强化学习中的重要性。在 CartPole 问题上，PPO 算法在 1829 轮训练后达到平均得分 495.72 分，成功解决环境，验证了算法的有效性。

第二节 研究贡献

本研究的主要贡献体现在理论、实践和应用三个层面。在理论层面，提供了 PPO 算法的完整实现和详细分析，深入阐述了算法的核心机制和数学原理，为理解 PPO 算法提供了系统性参考。在实践层面，针对 CartPole 问题提出了优化的超参数配置方案，通过系统的实验对比确定了最优参数组合，为实际应用提供了可借鉴的经验。在应用层面，验证了 PPO 算法在连续控制任务中的有效性，证明了算法在解决实际控制问题方面的潜力，为类似强化学习问题提供了有价值的参考和借鉴。

第三节 未来展望

未来的研究方向可以从算法扩展、参数优化、多智能体协作和样本效率等多个维度展开。首先，将 PPO 算法扩展到更复杂的连续控制任务，如机器人运动控制和自动驾驶，验证算法在真实复杂环境中的适用性和鲁棒性。其次，探索自适应超参数调整机制，通过元学习或自动机器学习技术实现参数的自动优化，提高算法的通用性和易用性。再次，研究多智能体 PPO 算法在协作任务中的应用，解决多智能体协调控制和博弈问题，拓展算法的应用范围。最后，结合模型预测控制和世界模型，提高样本效率，减少训练所需的交互次数，降低实际应用中的成本和时间消耗。

符号、标志、缩略语注释表

符号/缩略语	含义
PPO	近端策略优化
GAE	广义优势估计
MDP	马尔可夫决策过程
γ	折扣因子
λ	GAE 参数
ϵ	裁剪参数
π_θ	参数化策略
$A(s, a)$	优势函数
$V(s)$	状态价值函数
$r_t(\theta)$	重要性采样比率

附录

附录 A: PPO 算法核心代码

Listing 1: PPO 算法核心实现

```
1 class PPO:
2     def __init__(self, state_dim, action_dim, lr=2.5e-4,
3                 gamma=0.99, K_epochs=10, eps_clip=0.2,
4                 gae_lambda=0.95):
5         self.gamma = gamma
6         self.eps_clip = eps_clip
7         self.K_epochs = K_epochs
8         self.gae_lambda = gae_lambda
9
10        self.policy = ActorCritic(state_dim, action_dim)
11        self.optimizer = optim.Adam(
12            self.policy.parameters(), lr=lr, eps=1e-5)
13
14        self.policy_old = ActorCritic(state_dim, action_dim)
15        self.policy_old.load_state_dict(
16            self.policy.state_dict())
17
18        def compute_gae(self, rewards, values, dones, next_value):
19            advantages = []
20            gae = 0
21            for t in reversed(range(len(rewards))):
22                if t == len(rewards) - 1:
23                    next_val = next_value
24                else:
25                    next_val = values[t + 1]
26
27                delta = rewards[t] + self.gamma * next_val * \
28                    (1 - dones[t]) - values[t]
29                gae = delta + self.gamma * self.gae_lambda * \
30                    (1 - dones[t]) * gae
31                advantages.insert(0, gae)
32            return advantages
```

附录 B：训练参数配置

表 2: 完整训练参数配置

参数	值
环境	CartPole-v1
最大训练轮数	2000
最大步数	500
更新步数	2048
打印频率	20
解决标准	平均得分 ≥ 495